



opti_route

Plateforme mobile d'optimisation de tournées pour chauffeurs-livreurs indépendants
— rapport produit, 11 mai 2026

Synthèse exécutive

opti_route est une application Android destinée aux chauffeurs-livreurs indépendants, aux auto-entrepreneurs en livraison à domicile et aux coursiers urbains. Elle remplace les feuilles Excel, les plannings papier et les abonnements logiciels coûteux (50 à 150 € par mois) par un outil mobile complet, gratuit et fonctionnant sans abonnement.

Le produit existe en version fonctionnelle et testée. Il est en phase de pré-publication Play Store. La conception, le développement, le design graphique et l'infrastructure documentaire sont déjà en place. La première publication peut intervenir dans les semaines qui viennent, sous réserve de la création d'un compte développeur Google (paiement unique de 25 USD).

Les trois piliers fonctionnels qui structurent l'application sont les suivants. D'abord, l'optimisation d'itinéraire (calcul de l'ordre des arrêts, tracé d'itinéraire, gestion des contraintes terrain). Ensuite, le carnet d'adresses client local avec statistiques et historique. Enfin, le scan automatisé des bordereaux de livraison par reconnaissance optique de caractères, qui élimine la saisie manuelle d'adresses la plus chronophage du métier.

L'architecture est volontairement locale (la base de données est sur le téléphone, jamais sur un serveur distant). Cette approche garantit la confidentialité des données client de chaque livreur et permet de fonctionner sans coût d'infrastructure récurrent. Les services externes utilisés sont tous des API publiques gratuites adossées à des organismes publics français ou à des projets open source matures.

Problème adressé

Le chauffeur-livreur indépendant est confronté à plusieurs tensions structurelles dans son quotidien.

Premièrement, l'organisation de sa tournée est encore largement manuelle dans la grande majorité des cas. Le livreur reçoit une liasse de bordereaux le matin, parfois cinquante ou plus, et doit décider seul de l'ordre dans lequel les traiter. Un mauvais classement représente plusieurs dizaines de kilomètres et une à deux heures perdues par jour. Sur une année, le coût direct (carburant et temps de travail non facturé) est significatif.

Deuxièmement, la saisie des arrêts dans les outils logistiques existants reste laborieuse. Pour un bordereau classique, il faut taper manuellement le nom du destinataire, son adresse complète, le code postal, la ville, le nombre de colis. Un livreur qui traite cinquante arrêts par jour consacre entre trente et quarante-cinq minutes à cette ressaisie. Cette tâche est répétitive, sans valeur ajoutée et source d'erreurs (adresses fausses, ratés du géocodage).

Troisièmement, les solutions de marché existantes ciblent principalement les flottes professionnelles. Elles facturent des abonnements mensuels indexés sur le nombre de véhicules, demandent une carte bancaire à l'inscription, et imposent une remontée systématique des données sur des serveurs centralisés. Cette modèle est inadapté au livreur indépendant, qui n'a ni le budget récurrent, ni le souhait de remonter ses données clients à un tiers commercial.

Quatrièmement, le livreur indépendant a peu de levier pour conserver et exploiter sa connaissance terrain accumulée. Les adresses des clients livrés régulièrement, leurs particularités (code d'accès, étage, fenêtres horaires), les raisons d'échec récurrentes, tout cela disparaît à la fin de la journée sauf à le coucher dans un carnet papier ou un tableur informel.

Proposition de valeur

opti_route répond à ces quatre tensions par une réponse intégrée et locale.

Sur l'organisation de la tournée, l'application calcule l'ordre optimal des arrêts en quelques secondes en s'appuyant sur OpenRouteService, un service européen alimenté par OpenStreetMap et utilisant le moteur d'optimisation VROOM. Le livreur peut ensuite réordonner manuellement les arrêts pour corriger les sens uniques mal mappés ou intégrer des contraintes que le solveur ne peut pas modéliser (places de stationnement difficiles, demi-tours impossibles pour un fourgon, etc.).

Sur la saisie des arrêts, l'application intègre un scanner OCR alimenté par Google ML Kit. Une photo du bordereau suffit pour extraire automatiquement le nom du destinataire, l'adresse complète, le nombre de colis et le téléphone éventuel. Le format MESEXP (Messagerie Express, standard utilisé par de nombreux transporteurs français) est nativement supporté. Le traitement OCR est effectué entièrement sur le téléphone, sans appel à un service cloud, et fonctionne donc même sans réseau.

Sur le modèle économique, l'application est gratuite et le restera dans sa fonctionnalité principale. Les services externes utilisés (BAN, Recherche-Entreprises, OpenRouteService) ont tous un palier gratuit suffisant pour un usage individuel intensif. OpenRouteService autorise 500 optimisations par jour, ce qui couvre largement le besoin du livreur le plus actif. Aucune carte bancaire n'est requise pour utiliser l'application, ni de l'utilisateur, ni du développeur (à l'exception du paiement unique de 25 USD pour ouvrir un compte Play Console).

Sur la connaissance terrain, opti_route maintient automatiquement un carnet d'adresses local. Chaque arrêt validé enrichit ce carnet (nom client, géolocalisation, statistiques d'utilisation). Les arrêts ultérieurs proposent une auto-complétion sur les clients déjà connus, ce qui économise la ressaisie. Le livreur peut épingler ses clients critiques en favoris, leur attribuer des couleurs de repérage, consulter les statistiques par client (nombre de livraisons réussies et raisons d'échec). Une version PDF imprimable de ce carnet peut être générée à tout moment.

Différenciateurs

Le produit présente plusieurs points de différenciation par rapport aux solutions de marché existantes.

Premièrement, l'architecture local-first. La base de données SQLite réside sur le téléphone de l'utilisateur. Aucune donnée commerciale (nom client, adresse, historique de livraison) ne transite par un serveur opti_route, car aucun serveur opti_route n'existe. Cette propriété est techniquement vérifiable dans le code source et est explicitement énoncée dans la politique de confidentialité. Elle constitue un argument fort à la fois pour la confiance utilisateur et pour la conformité RGPD : il n'y a tout simplement aucune donnée personnelle traitée par l'éditeur de l'application.

Deuxièmement, l'absence totale de traceurs commerciaux. L'application n'intègre ni régie publicitaire, ni outil d'analyse, ni identifiant publicitaire, ni cookies, ni système de fingerprinting. Le suivi des plantages techniques sera assuré par Android Vitals, qui est intégré à Google Play Console sans nécessiter de SDK tiers dans le binaire. Ce positionnement clean est devenu rare et représente un signal de confiance dans un écosystème mobile saturé de SDK opaques.

Troisièmement, le scan OCR des bordereaux est un point différenciant fort vis-à-vis des concurrents généralistes. La plupart des solutions de planification logistique attendent que l'utilisateur saisisse manuellement ses arrêts ou les importe depuis un fichier CSV fourni par un donneur d'ordres. opti_route accepte la matière première réelle du livreur indépendant : la liasse de bordereaux papier qu'il reçoit chaque matin.

Quatrièmement, la palette ergonomique a été pensée pour la conduite. La typographie principale est Manrope, la palette est en cream et ink avec des accents lime et emerald, le contraste a été calibré pour la lecture en plein soleil. Un mode plein écran sans barres système est disponible pour la consultation de la carte au volant. Le mode sombre est en cours de finalisation pour la conduite de nuit.

Architecture technique

L'application est développée en Flutter (Dart), ce qui garantit la possibilité d'un portage iOS ultérieur à coût marginal, et donne accès à un écosystème de packages mature.

La persistance des données est assurée par Drift, un ORM SQLite type-safe en Dart, avec un schéma actuellement à la version 15. Sept tables structurent les données : tournées, arrêts, paramètres utilisateur, scans de bordereaux, cache de géocodage, carnet d'adresses, historique de transitions de statut. Le schéma comporte des contraintes de clé étrangère avec cascade pour garantir l'intégrité référentielle.

La gestion d'état utilise Riverpod, ce qui permet une séparation claire entre la couche de données et la couche d'affichage, et facilite l'écriture de tests unitaires sur la logique métier.

L'optimisation d'itinéraire est déléguée à OpenRouteService (API publique européenne, plan gratuit 500 requêtes par jour, sans CB). Le cas où le quota est dépassé est géré : l'utilisateur peut alors organiser ses arrêts manuellement par drag-and-drop, l'application reste fonctionnelle.

Le géocodage des adresses procède par cascade en commençant par BAN (Base Adresse Nationale, donnée publique data.gouv.fr) pour les adresses postales françaises. Si la requête commence par un nom d'entreprise plutôt qu'un numéro, la cascade interroge d'abord Recherche-Entreprises (annuaire SIRENE public) pour matcher par enseigne. Photon (service Komoot basé sur OpenStreetMap) sert de fallback international pour les marques et chaînes connues.

La cartographie utilise flutter_map avec les tuiles OpenStreetMap. Un cache disque local conserve les tuiles déjà téléchargées dans le dossier de cache de l'application, ce qui réduit la consommation de données et permet un usage partiellement hors-ligne dans les zones déjà visitées.

L'OCR est confié à Google ML Kit Text Recognition. Le modèle de reconnaissance est local au téléphone, aucun appel cloud n'est fait. Un parser dédié (BordereauParser) interprète le texte brut extrait par ML Kit pour structurer les champs métier en s'appuyant sur des heuristiques de positionnement et de fréquence (le nom du destinataire apparaît deux fois sur un bordereau MESEXP, le nom de l'expéditeur une seule fois, ce qui permet de les distinguer automatiquement).

Les notifications locales sont gérées par flutter_local_notifications avec planification exacte. Un canal Android dédié regroupe les rappels de tournée. Aucune notification push provenant d'un serveur distant n'est utilisée.

L'application est packagée en Android App Bundle pour la distribution Play Store, signée avec une keystore release dont la procédure de génération est documentée. Le build de release peut être effectué localement par un simple `flutter build appbundle --release`.

Tests et qualité

La couverture de tests automatisés est de 119 tests Dart au moment de la rédaction. Cette base couvre les briques métier sensibles : optimisation et ordre des priorités, repository des arrêts et transitions de statut, export CSV au format RFC 4180, import CSV avec fusion par upsert, calcul des statistiques cumulatives et journalières, parsing des bordereaux, suggestion d'adresse, cycle pause et reprise du chronomètre de tournée, calcul de l'ETA par arrêt, géocodage SIRENE.

Le pipeline de CI s'exécute automatiquement sur chaque pull request via GitHub Actions. Il enchaîne la résolution des dépendances Flutter, la régénération du code Drift, l'analyse statique stricte (`flutter analyze`) et l'exécution complète de la suite de tests. Aucun warning n'est toléré.

La discipline de développement utilise un workflow trunk-based avec pull requests squashées. Les commits suivent la convention Conventional Commits (`feat:`, `fix:`, `chore:`, etc.). Chaque pull request est documentée par un titre court et un résumé d'intention.

Fonctionnalités produit

Cette section détaille l'inventaire des fonctionnalités opérationnelles, regroupées par domaine.

Création et planification de tournée

L'utilisateur crée une tournée en saisissant un nom, une date et un point de départ. Le point de départ est géocodé automatiquement à partir d'une adresse libre. La capacité du véhicule en colis peut être renseignée pour activer une alerte si l'ensemble des arrêts dépasse cette capacité.

Les arrêts sont ajoutés un par un (manuellement) ou par scan de bordereau. Chaque arrêt comporte une adresse, un nom client optionnel, un nombre de colis, une durée d'arrêt estimée, et optionnellement une fenêtre horaire (par exemple disponibilité uniquement l'après-midi).

Les arrêts peuvent être marqués comme "obligatoire en premier" ou "obligatoire en dernier", ce qui contraint l'ordre dans le solveur. Un arrêt marqué "à éviter si possible" voit sa priorité abaissée et sera placé en dernier en cas de saturation horaire.

Optimisation

L'utilisateur déclenche l'optimisation par un bouton dédié. L'application transmet la liste des arrêts géolocalisés à OpenRouteService et reçoit en retour l'ordre optimal, la distance totale, la durée totale et le tracé géographique de l'itinéraire.

Le bouton "Optimiser" est automatiquement désactivé si aucune modification structurelle n'a eu lieu depuis la dernière optimisation, ce qui évite la consommation inutile du quota quotidien. Toute modification d'un arrêt (ajout, suppression, changement d'adresse) ré-active le bouton.

Le tracé d'itinéraire est conservé en base et affiché sur la carte sous forme de polyline.

Un compteur d'optimisations consommées dans la journée est affiché dans les paramètres, avec réinitialisation automatique à minuit.

Réordonnancement manuel

Après optimisation, le livreur peut intervenir manuellement sur l'ordre des arrêts via un mécanisme de drag-and-drop. Cette possibilité est essentielle car l'optimiseur ne peut pas modéliser toutes les contraintes terrain (sens uniques inexacts dans OpenStreetMap, place de stationnement difficile, demi-tour impossible pour un fourgon).

Le mécanisme conserve les contraintes "premier" et "dernier" : seul le bloc intermédiaire est réordonnable, sauf si l'utilisateur lève explicitement les contraintes.

Templates récurrents

Une tournée existante peut être marquée comme template (tournée modèle). Elle apparaît alors épinglée en haut de l'historique avec un bouton "Créer une tournée depuis ce template". L'action duplique l'arborescence complète (arrêts inclus) en remettant les statuts à zéro. Ce mécanisme est conçu pour les livreurs ayant des tournées hebdomadaires récurrentes (par exemple les mêmes trente clients chaque mercredi).

Multi-tournées par jour

L'application permet de planifier plusieurs tournées sur la même date (par exemple une tournée du matin et une tournée de l'après-midi). Un bandeau "X autres tournées aujourd'hui" permet de switcher rapidement entre elles depuis l'écran d'accueil.

Exécution de tournée

Au démarrage de la tournée, l'utilisateur tape sur "Démarrer". Le statut bascule en `en\_cours`, le

chronomètre s'enclenche, et le pin "moi" devient visible sur la carte (cercle bleu à la position GPS courante).

Un bouton de pause permet de suspendre le chronomètre. La durée totale des pauses est cumulée séparément, ce qui permet d'afficher le temps actif réel (hors pauses) en fin de tournée. Cette donnée est utile pour la facturation à l'heure ou pour analyser sa productivité.

La liste des arrêts est filtrable par statut (tous, à livrer, livrés, échecs) et triable par distance GPS si l'utilisateur s'est écarté de l'itinéraire prévu. Une heure d'arrivée estimée par arrêt est calculée et affichée sous la forme "Arrivée vers 14h27". Le calcul utilise la distance haversine cumulée depuis le dernier arrêt validé (ou le point de départ) combinée à une vitesse moyenne déduite des données ORS. La précision typique est de plus ou moins quinze pourcent, suffisante pour donner un ordre de grandeur.

Validation des arrêts

Au moment de la livraison, un tap sur l'arrêt ouvre une bottom sheet de validation. L'utilisateur peut marquer l'arrêt comme livré, en échec avec choix d'une raison (absent, refusé, adresse fausse, autre), ou revenir à l'état "à livrer" pour annuler. Une capture GPS est effectuée au moment de la validation et persistée en base comme preuve de passage en cas de litige client.

L'édition rapide du nombre de colis et des notes est possible directement dans la bottom sheet, sans avoir à ouvrir l'écran d'édition complet. Les notes bénéficient d'une auto-sauvegarde après six cents millisecondes d'inactivité. Un rappel discret sous le champ indique que la dictée vocale Android est accessible par appui long sur la barre d'espace, ce qui permet une utilisation à une main au volant.

Un bouton "Tout marquer livré" est disponible dans le menu de l'AppBar pour les cas où plusieurs colis sont livrés en bloc à la même adresse ou à un point de collecte.

Chaque transition de statut est enregistrée dans une table d'audit séparée (action, statut avant, statut après, raison éventuelle, timestamp), ce qui constitue une piste d'audit complète exploitable en cas de litige.

Vue carte

L'application affiche une carte OpenStreetMap centrée sur la zone de la tournée. Le dépôt est représenté par un pin lime, les arrêts par des pins numérotés colorés selon leur statut (blanc pour à livrer, vert pour livré, rouge pour échec). Le tracé de l'itinéraire optimisé est superposé en polyline verte lorsqu'il est disponible.

Un mode plein écran masque les barres système Android pour offrir une vue maximale lors de la conduite. Un bouton flottant permet de revenir au mode standard.

Pendant l'exécution d'une tournée, un pin "moi" matérialisé par un cercle bleu plein avec halo translucide indique la position GPS en temps réel. Ce pin n'est affiché que pendant les tournées actives, jamais sur les brouillons ou les tournées terminées.

Un tap sur un pin d'arrêt ouvre une bottom sheet d'information (nom client, adresse, notes, nombre de colis, fenêtre horaire éventuelle).

Carnet d'adresses

Chaque arrêt validé enrichit automatiquement le carnet d'adresses local. La logique de dédoublonnage repose sur le nom du client (insensible à la casse) ou sur la proximité des coordonnées (rayon d'environ cent dix mètres). Un compteur d'utilisations et une date de dernière visite sont maintenus par entrée.

Le livreur peut consulter le carnet depuis le drawer principal. La recherche supporte la normalisation des accents et matche sur le nom client, l'adresse et la ville.

Chaque entrée peut être éditée individuellement. Un toggle "favori" remonte l'entrée en haut de la liste indépendamment du nombre d'utilisations, ce qui est utile pour les clients critiques ou à soigner. Une couleur de repérage choisie parmi six (lime, emerald, red, amber, cream, ink) personnalise la pastille de l'entrée pour le repérage visuel rapide dans une longue liste.

Un panneau de statistiques par client est accessible : nombre de livraisons réussies, nombre d'échecs,

date de dernière visite, top trois des raisons d'échec triées par fréquence.

L'import et l'export CSV au format RFC 4180 permettent la sauvegarde et la migration entre appareils. Un export PDF du carnet sous forme d'annuaire imprimable est également disponible, avec les favoris regroupés en haut et tri alphabétique du reste. Une bannière passive apparaît en tête de la liste si plus de quatorze jours se sont écoulés depuis le dernier export, pour rappeler à l'utilisateur de sauvegarder ses données.

Scan OCR des bordereaux

L'utilisateur peut photographier un bordereau depuis l'application. La photo est traitée localement par Google ML Kit Text Recognition (modèle on-device, aucun appel cloud). Le texte brut extrait alimente un parser dédié qui interprète la structure.

Le format MESEXP (Messagerie Express, standard utilisé par de nombreux transporteurs français) est nativement supporté. Les champs extraits sont le nom du destinataire, la rue, le code postal, la ville, le nombre de colis et le téléphone s'il est présent.

Le parser inclut des heuristiques pour tolérer les fautes d'OCR mineures (caractère manquant dans un mot-clé, espace inséré dans une référence), les photos prises tête-bêche (ML Kit retourne les lignes dans un ordre chaotique sur ce format), et la distinction entre expéditeur et destinataire (qui apparaît deux fois sur le bordereau, contrairement à l'expéditeur qui n'apparaît qu'une fois).

Un score de confiance est calculé pour chaque extraction. Un indicateur visuel sur l'écran d'ajout d'arrêt signale par une carte orange les bordereaux ambigus pour lesquels l'utilisateur doit vérifier manuellement les champs avant validation.

Un parser dédié aux bordereaux "collés sur les colis" (autre format reporté par le terrain) est en cours de développement et sera ajouté dès que les photos de référence auront été transmises.

Statistiques

Un écran de statistiques cumulatives est accessible depuis le drawer principal. Trois fenêtres de temps sont calculées : les sept derniers jours, les trente derniers jours, et les trois cent soixante-cinq derniers jours. Pour chaque fenêtre, l'application affiche le nombre de tournées planifiées et terminées, le nombre d'arrêts, le nombre de colis livrés, la distance totale parcourue, la durée totale et le taux de réussite (livrés sur tentatives).

Un mini graphique en barres est affiché en tête de l'écran, montrant les colis livrés par jour sur les quatorze derniers jours. Les barres sont en émeraude, avec la barre du jour courant en lime pour la distinguer. Une pastille rouge surmonte les barres des journées ayant comporté au moins un échec.

Notifications locales

L'application supporte les notifications locales planifiées. Un bouton de test dans les paramètres permet de vérifier que la chaîne (permission Android, canal de notification, planification exacte) fonctionne correctement avant un usage en production.

Un rappel quotidien optionnel peut être activé via un toggle dans les paramètres. Lorsqu'il est actif, une notification est planifiée chaque soir à dix-neuf heures pour rappeler à l'utilisateur de vérifier sa tournée du lendemain. La planification utilise le mécanisme natif de récurrence quotidienne d'Android.

Paramètres et administration

Les paramètres regroupent la configuration de l'application : clé API OpenRouteService (saisie masquée, stockée localement), capacité véhicule par défaut, durée d'arrêt par défaut, application de navigation préférée (Google Maps ou Waze).

Une zone de maintenance permet de surveiller le compteur d'optimisations consommées dans la journée, de nettoyer les tournées de plus d'un an, de vider le cache des tuiles cartographiques, et de vider le cache du géocodage.

Une zone de notifications regroupe le test ponctuel et le toggle de rappel quotidien.

Un menu d'apparence permet de choisir entre le mode système, le mode clair et le mode sombre.

Une zone "à propos" donne accès aux mentions légales (politique de confidentialité et conditions générales d'utilisation).

Confidentialité et stockage

Toutes les données de l'utilisateur sont stockées localement dans une base SQLite sur le téléphone. Aucun serveur opti_route n'existe et aucune donnée ne lui est jamais transmise.

Les permissions Android demandées sont au nombre de quatre : Internet (pour les API publiques de géocodage et d'optimisation), Caméra (pour le scan de bordereaux avec traitement OCR local), Localisation (uniquement pendant le mode tournée en cours, jamais en arrière-plan), Notifications (pour les rappels locaux).

L'utilisateur peut révoquer la permission de localisation depuis les paramètres Android à tout moment. L'application reste fonctionnelle sans GPS live (la carte n'aura simplement pas de pin "moi").

La désinstallation de l'application supprime intégralement la base SQLite locale et donc toutes les données utilisateur.

Export et partage

L'application permet de générer un PDF récapitulatif d'une tournée, comprenant l'en-tête (nom de tournée, date, dépôt), les statistiques (kilomètres, durée, taux de réussite), et le tableau des arrêts avec leur statut et leur éventuelle raison d'échec. Le PDF est ensuite partagé via le sélecteur natif Android, ce qui permet l'envoi par WhatsApp, par mail, ou vers Google Drive.

Le carnet d'adresses peut être exporté en CSV (sauvegarde complète, formats lisibles dans Excel ou LibreOffice) ou en PDF (annuaire imprimable).

Les actions de partage sont déclenchées uniquement par l'utilisateur via des boutons dédiés. Aucun partage automatique ou en arrière-plan n'est effectué.

État du produit

Le produit est en phase de pré-publication Play Store. La quasi-totalité des fonctionnalités décrites ci-dessus sont codées et testées. Trois branches de développement sont actuellement en attente de revue et de merge sur la branche principale du dépôt : la première regroupe les améliorations ergonomiques de terrain (badge "en cours", ETA par arrêt, copie d'adresse, fond teinté par statut, suite de tests), la deuxième regroupe les nouveaux écrans (mini bar chart, pin "moi" live, bannière de sauvegarde, rappel quotidien, bouton pause), la troisième regroupe la documentation de publication Play Store (description, screenshots, procédure pas-à-pas).

Au total, quinze commits sont prêts à être mergés. La suite de tests passe au complet (cent dix-neuf tests verts) et l'analyseur statique ne signale aucun problème.

L'icône de l'application et le splash screen sont finalisés, basés sur un design réalisé avec Claude Design (pin lime et trace de route sur fond ink). Les assets sont versionnés dans le dépôt aux différentes tailles requises par Android.

La politique de confidentialité et les conditions générales d'utilisation sont rédigées en français et prêtes à être hébergées sur GitHub Pages.

La description Play Store est rédigée (titre, description courte, description longue de trois mille huit cent cinquante caractères) et prête à être copiée dans la console.

La procédure de publication est documentée pas-à-pas dans le dépôt, y compris les pièges connus (URL de politique de confidentialité inaccessible, icône floue, screenshots mockés rejetés par Google).

Roadmap

La roadmap court terme (semaines à venir) couvre la finalisation du mode sombre sur l'ensemble des écrans, la complétion du parser de bordereaux pour le format "collés sur les colis", et la publication effective sur le Play Store en piste de test interne.

La roadmap moyen terme (trois à six mois) couvre l'ajout d'un module de navigation turn-by-turn intégré pour éviter à l'utilisateur de basculer entre opti_route et Google Maps ou Waze. La piste prioritaire est l'intégration du SDK Mapbox (plan gratuit cinquante mille chargements de carte par mois, ce qui couvre largement un usage individuel). Cette feature transforme l'application d'un outil de planification en une suite complète de gestion de tournée.

La roadmap moyen terme couvre également un mode hors-ligne complet avec pré-cache d'une zone définie (tuiles et routes), pour les livreurs intervenant en zones rurales mal couvertes par le réseau.

La roadmap moyen terme couvre enfin un mode multi-livreurs en lecture seule pour les utilisateurs qui prennent un apprenti ou un coéquipier. Ce mode permet le partage d'une tournée via un lien local (par exemple QR code généré par le téléphone du chef et scanné par celui de l'apprenti), sans nécessiter de backend.

La roadmap long terme (six à douze mois) couvre l'ouverture éventuelle d'une version freemium pour les utilisateurs souhaitant lever certaines limites (quotas API supérieurs, sauvegarde cloud chiffrée optionnelle, support multi-appareils). La fonctionnalité de base resterait gratuite et non dégradée. Cette ouverture n'est pas une priorité immédiate, l'objectif initial étant de stabiliser le produit gratuit et de constituer une base d'utilisateurs avant d'introduire un palier payant.

La roadmap long terme couvre également un portage iOS, rendu peu coûteux par le choix de Flutter. La principale difficulté sera la régénération des assets aux dimensions iOS et la traversée du processus de validation App Store, plus exigeant que Google Play.

Marché et concurrence

Le marché cible primaire est le livreur indépendant français, regroupant les auto-entrepreneurs en livraison à domicile, les coursiers urbains et ruraux, les chauffeurs sous-traitants des grands transporteurs (Geodis, Chronopost, Colissimo, etc.) qui ne disposent pas de leur propre outil de tournée fourni.

Le marché cible secondaire est constitué des très petites flottes (un à cinq véhicules) qui sont actuellement sous-servies par les solutions de marché trop coûteuses ou trop complexes pour leur taille.

La concurrence directe se segmente en trois catégories. Les solutions professionnelles haut de gamme (Onfleet, Routific, Locus) facturent entre cinquante et plus de deux cents euros par véhicule par mois, ciblent les flottes structurées et imposent un onboarding payant. Les applications grand public (Google Maps, Waze) ne font pas d'optimisation de tournée multi-arrêts et n'ont pas de notion de carnet d'adresses client. Les solutions intermédiaires (Circuit, Speedy Route) sont positionnées sur du dix à vingt euros par mois et ciblent partiellement le segment indépendant, mais conservent un modèle d'abonnement et une politique de données moins favorable.

Le positionnement d'opti_route est de combiner la qualité fonctionnelle des solutions professionnelles (optimisation, carnet, OCR) avec le modèle gratuit et la philosophie local-first des outils grand public. Ce positionnement n'est, à ma connaissance, pas occupé par un acteur établi en France.

Modèle économique

À l'horizon court et moyen terme, le modèle est entièrement gratuit, sans publicité et sans collecte de données. Le coût marginal par utilisateur supplémentaire est nul pour l'éditeur, puisque les services externes utilisés ont un quota par utilisateur final (chacun crée sa propre clé OpenRouteService).

À l'horizon long terme, plusieurs pistes de monétisation non intrusives sont envisageables et seraient déclenchées uniquement si une base utilisateur significative se constitue.

Une première piste est un palier "pro" payant facultatif (par exemple cinq à dix euros par mois) débloquent des fonctionnalités à coût d'infrastructure pour l'éditeur (sauvegarde cloud chiffrée, synchronisation multi-appareils, quotas API rehaussés via une clé partagée). La fonctionnalité de base resterait gratuite et non dégradée.

Une deuxième piste est une version "équipe" pour les petites flottes (cinq à vingt véhicules) facturée à l'utilisateur, qui ajouterait un tableau de bord centralisé en lecture seule et une attribution de tournées entre livreurs.

Une troisième piste est un partenariat avec des transporteurs ou des éditeurs de logiciels métier complémentaires (comptabilité, facturation auto-entrepreneur) pour la mise en place de connecteurs spécifiques, facturés au partenaire et non à l'utilisateur final.

Ces pistes sont mentionnées à titre exploratoire. Aucune n'est en chantier actif, et la priorité reste la constitution d'une base utilisateur fidèle sur le produit gratuit.

Équipe et exécution

Le produit a été conçu, développé et déployé par Noah Trillon (chauffeur-livreur en activité, qui a identifié le besoin sur le terrain et le formalise depuis plusieurs mois) en collaboration avec un outil de génération assistée pour le code, le design et la documentation.

Le design graphique (icône, splash screen, palette, typographie) a été réalisé en concertation avec Claude Design, avec un handoff complet versionné dans le dépôt.

L'infrastructure de développement (intégration continue GitHub Actions, conventions de commit, pull request workflow, suite de tests, documentation projet) est de niveau professionnel et permet une reprise du projet par un développeur tiers si nécessaire.

Annexe — limites connues et points d'attention

Cette section liste honnêtement les limites actuelles du produit, afin de ne pas sur vendre.

Le mode sombre n'est complet qu'à environ soixante-dix pourcent des écrans. Certains éléments restent figés en mode clair. Un refactor complet est planifié pour la phase de pré-publication finale.

L'optimisation d'itinéraire ne modélise pas tous les détails du terrain (places de stationnement, demi-tours fourgon, sens uniques mal renseignés dans OpenStreetMap). Le mécanisme de drag-and-drop manuel a été ajouté précisément pour pallier ces limites.

Le scan OCR fonctionne bien sur le format MESEXP. D'autres formats (bordereaux collés sur les colis, formats spécifiques à certains transporteurs) nécessitent un travail d'extension dédié, en attente des photos de référence du terrain.

Le quota d'optimisations OpenRouteService est de cinq cents par jour sur le plan gratuit. Cette limite est largement suffisante pour un usage individuel intensif, mais pourrait être contraignante pour une montée en puissance vers le segment "petites flottes". Une bascule vers un plan payant ORS ou vers une autre infrastructure (VROOM auto-hébergé) serait alors envisageable.

L'application est aujourd'hui exclusivement Android. Le portage iOS est techniquement faisable (Flutter) mais demande un compte développeur Apple (cent dollars par an, contrairement aux vingt-cinq dollars unique d'Android) et un effort de validation supplémentaire.

L'auteur est seul sur le développement et le support utilisateur. Un système de bug report par email est en place, mais la disponibilité de réponse n'est pas garantie. Cette limite pourrait devenir critique en cas de croissance utilisateur rapide.

Contact

Pour toute question, demande de démonstration ou échange exploratoire : noah.trillon28@gmail.com

Le code source est disponible sur GitHub à l'adresse https://github.com/chipat-neko/opti_route et fait l'objet d'une licence à définir (envisagée : MIT pour le code, propriétaire pour la marque opti_route).